SEARCH-CONCENTRATION AUDITS FOR LEARNED-DYNAMICS TREE PLANNING

Anonymous authors
Paper under double-blind review

ABSTRACT

Tree search over learned dynamics is often treated as a clean way to spend more test-time computation. This paper audits a narrower and more dangerous possibility: adaptive tree expansion can concentrate model queries on a locally optimistic error, turning a learned simulator into its own adversarial objective. We construct a deterministic two-dimensional control audit with a known off-route bias pocket, compare static rollout selection, UCT MCTS, value-guided MCTS, uncertainty-penalized MCTS, and conservative-backup MCTS under equal forward-model budgets, and commit every numerical claim to generated CSV/JSON artifacts. The result is a tail-risk finding rather than a dominance claim. At budget 1024, UCT has a larger mean selected-return optimism gap than the static rollout pool (0.686 versus 0.301) and a larger worst branch-capture event (7.021 versus 3.301), but the paired median delta is approximately zero and UCT is worse on only 20% of paired seeds. A tail-stress suite replays the capture seeds, sweeps uncertainty penalties, and varies reward optimism; a strong uncertainty penalty collapses the largest replayed UCT event from 7.021 to 0.430. A second expansion suite attacks the claim from seven additional angles: exploration constants, horizon and budget, action libraries, uncertainty calibration, dynamics drift, start-state geometry, and receding-horizon replay. The v4 standard benchmark tier adds Gymnasium CliffWalking-v1 with a biased learned transition table: UCT amplifies the learned cliff-shortcut optimism gap by 165.38 relative to a behavioral static rollout pool, while uncertainty-penalized MCTS reduces the gap by 184.02 and improves true return by 166.82. The final claim is deliberately conditional: uncertainty can repair search concentration when its signal is aligned and the penalty is calibrated, but it is not a general guarantee.

1 INTRODUCTION

Learned model-based reinforcement learning promises a useful bargain: if a model of the environment is good enough, an agent can spend additional computation imagining futures instead of paying for more real interaction. PETS, PlaNet, Dreamer, and related systems show that learned dynamics can support planning or policy improvement from limited data (Chua et al., 2018; Hafner et al., 2019; 2020). Tree-search methods make the bargain sharper. Rather than evaluating a fixed batch of candidate rollouts, the planner allocates computation adaptively to branches that look promising under the model (Kocsis & Szepesvári, 2006; Browne et al., 2012; Schrittwieser et al., 2020; Kohankhaki et al., 2024; Riviere et al., 2024).

This adaptivity is useful when the model is a reliable evaluator over the region discovered by search. It is also exactly the feature that can amplify local model errors. If a learned dynamics or reward model is optimistic in a small off-route region, then a branch that accidentally enters that region can receive higher backed-up value, attract more visits, and receive still more model queries. The problem is not simply that a biased model is sampled. The problem is that the search rule can turn a sampled error into an allocation target.

We study that failure mode as a *search-concentration audit*. The audit is intentionally controlled. The environment is a deterministic point-mass task with a known reward and a known transition rule. The surrogate model is accurate almost everywhere but contains a localized optimism pocket: it predicts extra reward and a transition drift toward the goal near an off-route region. The planner is evaluated by taking the action sequence it selects under the surrogate and replaying exactly that sequence in the true environment. The primary metric is the selected-return optimism gap,

$$\Delta_{\mathrm{opt}}(a_{0:H-1}) = \hat{G}(a_{0:H-1}) - G(a_{0:H-1}),$$

where lower values are better. A large positive value means that the selected sequence looked much better under the learned simulator than it actually was.

The baseline is not a weak planner. It is a static rollout pool that samples independent open-loop sequences, scores each sequence once under the same surrogate, and executes the best first action. Static selection can still pick an optimistic sequence; it is not immune to model bias. What it cannot do is recursively allocate more model calls to a branch because earlier calls on that branch were optimistic. The comparison therefore separates *selection under a biased evaluator* from *adaptive allocation feedback under a biased evaluator*.

The evidence supports a narrow claim. In the full 20-seed run, UCT MCTS has a higher mean optimism gap and a much larger worst event than the static rollout pool at budget 1024. But UCT is not worse on most paired seeds. Its median paired delta is approximately zero, and only 20% of paired seeds have larger UCT gaps than static rollout. The danger is a rare heavy tail, not a broad ordering of planners. This distinction matters because a reviewer could otherwise reject the paper for overclaiming a dominance result that the data do not support.

The paper then attacks the mechanism in two ways. The tail-stress suite replays the capture seeds, sweeps uncertainty penalties, and varies the reward optimism injected into the pocket. This suite shows that a calibrated uncertainty penalty can suppress the largest replayed capture event. The expansion suite is more adversarial: it changes exploration constants, horizon, budget, action library, uncertainty strength, transition drift, start-state geometry, and receding-horizon execution. One expansion slice finds that the same strong uncertainty penalty can backfire at a reduced budget, producing a larger tail than UCT. That negative result is included because it prevents a second overclaim: uncertainty penalties are not magic; they are control knobs whose safety depends on calibration and search dynamics.

The contributions are:

- A controlled learned-dynamics tree-search audit that isolates adaptive allocation feedback as a mechanism for amplifying local model optimism.
- A comparison between static rollout selection and several MCTS variants under identical forward-model budgets, with tail quantiles and paired-seed deltas rather than only means.
- A tail-stress suite that replays branch-capture seeds, sweeps uncertainty penalties, and varies reward-bias severity.
- A seven-angle expansion suite covering exploration constants, horizon and budget, action-library granularity, uncertainty calibration, transition drift, start-state geometry, and receding-horizon replay.
- A Gymnasium CliffWalking-v1 benchmark (Towers et al., 2024) showing the same learned-shortcut failure under a standard tabular planning environment with an exact true transition table.
- A generated claim audit that marks every manuscript-level claim pass/fail and explicitly includes a repair-backfire counterexample.

2 Problem Statement And Audit Contract

2.1 Why A Controlled Audit?

The paper does not claim that a hand-designed point-mass task is a robotics benchmark. It is a microscope. The goal is to expose a mechanism clearly enough that the planner, model error, and selected failure event can be inspected. A benchmark result on a high-dimensional task would mix together model-learning quality, controller details, value-function extrapolation, action parameterization, and environment stochasticity. Those are important problems, but they are not the first question here. The first question is whether an adaptive planner can concentrate computation on a local model error in a way that a non-adaptive static rollout pool cannot.

The audit contract is therefore intentionally narrow:

- The true world is deterministic and known to the evaluator.
- The surrogate model is deliberately misspecified in a small region.
- Every planner is charged by forward-model calls.
- A planner is scored on the true return of the exact sequence it selected under the surrogate.
- Claims are made only when the generated claim audit passes.
- The paper reports paired deltas and tail quantiles, not only aggregate means.

This contract makes the experiment falsifiable. If UCT did not produce larger tail events than static rollout, the branch-capture mechanism would be unsupported. If uncertainty penalties helped only in the exact original setting but failed under perturbation, the repair claim would have to be narrowed. The final paper does exactly that: the mechanism claim survives the stress tests, while the repair claim becomes conditional.

2.2 Selected-Return Optimism

Let s_0 be the start state, $a_{0:H-1}$ an action sequence, P the true transition rule, and $\hat{P}$ the surrogate transition rule. Let r and $\hat{r}$ denote the true and surrogate rewards. The true and surrogate discounted returns for the same selected sequence are

$$G(a_{0:H-1}) = \sum_{t=0}^{H-1} \gamma^t r(s_{t+1}, a_t), \quad \hat{G}(a_{0:H-1}) = \sum_{t=0}^{H-1} \gamma^t \hat{r}(\hat{s}_{t+1}, a_t),$$

where $s_{t+1} = P(s_t, a_t)$ and $\hat{s}_{t+1} = \hat{P}(\hat{s}_t, a_t)$. The optimism gap is $\Delta_{\text{opt}} = \hat{G} - G$. Because the same action sequence is replayed in both systems, Δ_{opt} measures selected-sequence model optimism rather than a policy-learning objective.

This metric is stricter than checking the planner's internally backed-up value. A tree search value can be high because of backup noise, depth truncation, or value-estimator bias. The selected-return gap asks a simpler question: after the planner has chosen a sequence, how much better did that sequence look in the simulator than in the true world?

2.3 The Allocation Feedback Hypothesis

The hypothesis is not that MCTS is generally bad. UCT is an effective allocation rule when its evaluator is appropriate. The hypothesis is conditional:

$$\text{local model optimism} + \text{adaptive branch allocation} \quad \Longrightarrow \quad \text{rare high-optimism selected branches.}$$

The feedback loop has four stages. First, exploration reaches an optimistic region. Second, the surrogate gives that region higher reward or goal-directed dynamics than it deserves. Third, UCT backs up the optimistic value and increases the branch's visit share. Fourth, additional model calls are spent below that branch, increasing the chance that the final greedy sequence remains inside the error region. Static rollout selection can satisfy the first two stages but not the recursive third and fourth stages.

3 Environment, Model, and Planners

3.1 Point-Mass World

The true environment is a deterministic two-dimensional point robot. The state is a position $s \in [-1.05, 1.05]^2$. The start state is $(-0.85, -0.55)$ unless a stress test changes it. The goal is $(0.82, 0.55)$. Actions come from a discrete library and are scaled by a step size of 0.16. The true reward encourages progress to the goal, penalizes action cost, gives a terminal goal bonus inside a small radius, and applies a mild true penalty near an off-route pocket centered at $(-0.15, 0.82)$. The pocket penalty ensures that trajectories exploiting the pocket are not secretly good in the true world.

The world is small enough that failures can be inspected, but it has the ingredients needed for the mechanism: a goal-directed reward, a discrete planning problem, a misleading off-route region, and enough horizon for search to discover that region.

3.2 Biased Learned Model

The surrogate model matches the true transition almost everywhere. Near the pocket it adds two forms of optimism. The first is reward optimism:

$$\hat{r}(s, a) = r(\hat{s}, a) + \beta_r \, \rho(s),$$

where $\rho(s)$ is a Gaussian-shaped pocket score. The second is dynamics optimism:

$$\hat{s}' = s' + \beta_d \rho(s')(g - s'),$$

where g is the goal. Thus, entering the pocket makes the model believe the agent receives extra reward and drifts closer to the goal. The default parameters are $\beta_r = 1.15$ and $\beta_d = 0.22$.

The model also returns an uncertainty estimate,

$$u(s, a) = u_0 + u_p \rho(s') + \|\hat{s}' - s'\|_2.$$

This is deliberately favorable to uncertainty-aware repair because the uncertainty signal is aligned with the injected optimism pocket. If uncertainty repair fails even here, that failure should be taken seriously. If it succeeds here, the result should still not be interpreted as proof that arbitrary learned uncertainty estimates will be calibrated.

3.3 Planner Suite

All planners use the same horizon $H = 18$, discount $\gamma = 0.98$, action library, and forward-model accounting unless a stress test changes them.

Random open-loop. The random baseline samples one sequence and executes the first action. It is included only as a low-compute sanity check.

Static rollout pool. The static pool samples B/H independent open-loop sequences under budget B, scores each once under the surrogate, and selects the highest-scoring sequence. It is adaptive only at the final selection step; it never reallocates intermediate model calls based on earlier optimism.

UCT MCTS. UCT expands a tree over the learned model and selects child edges by

$$\text{score}(e) = \bar{Q}(e) + c\sqrt{\frac{\log(N_{\text{parent}} + 1)}{N_e}},$$

where c is the exploration constant. The final action sequence follows greedy child means from the root.

Value-guided MCTS. This variant adds a learned value estimate at leaves. The value estimate is also optimistic near the pocket, so it tests whether leaf value guidance changes the tail mechanism.

Uncertainty-penalized MCTS. This variant subtracts $\lambda u(s, a)$ during rollout scoring. The main full-run setting uses $\lambda = 0.85$; the tail-stress sweep also evaluates $\lambda = 2.40$, which is the strongest successful repair in the capture replay.

Conservative-backup MCTS. This variant subtracts the uncertainty penalty inside the backup reward. It tests whether penalizing the backed-up target differs from only penalizing rollout scoring.

Table 1: Expansion suite design. These tests broaden the paper beyond the initial 20-seed mechanism run without turning the artifact into a high-RAM benchmark job.

Stress family	Varied factors	Purpose
Exploration sweep	$c \in \{0.35, 0.75, 1.25, 2.0, 3.0\}$	Checks whether the UCT tail is a fixed-parameter accident.
Horizon/budget sweep	$H \in \{8, 18, 24\}$, $B \in \{256, 768\}$	Tests whether lookahead and compute change tail exposure.
Action-library sweep	cardinal4, default9, dense17	Tests whether branch geometry changes capture opportunities.
Uncertainty calibration	$u_p \in \{0, 0.15, 0.55, 1.10\}$	Attacks the repair claim directly.
Dynamics drift	$\beta_d \in \{0, 0.22, 0.36, 0.50\}$	Separates transition optimism from reward optimism.
Start-state geometry	base, lower, pocket-adjacent, wide-left	Tests geometric sensitivity of the pocket mechanism.
Closed-loop replay	3 receding-horizon steps, budget 512	Checks whether selected-plan optimism appears when replanning.

4 EXPERIMENTAL PROTOCOL

4.1 FULL MECHANISM RUN

The full mechanism run uses seeds $0, \ldots, 19$, budgets $\{64, 128, 256, 512, 1024\}$, and six planners: random open-loop, static rollout pool, UCT MCTS, value-guided MCTS, uncertainty MCTS, and conservative MCTS. The run writes plan metrics, closed-loop episode summaries for a small subset, depth profiles, root statistics, tail summaries, paired deltas, and figures under `results/full/` and `paper/figures/`.

4.2 TAIL-STRESS SUITE

The tail-stress suite is designed after observing the full-run failure mode but before writing the final claim ledger. It performs three tests at budget 1024:

- Capture replay over the same 20 seeds, comparing static rollout, UCT, two uncertainty penalties, and conservative backup.
- An uncertainty-penalty sweep over $\lambda \in \{0, 0.25, 0.50, 0.85, 1.25, 1.75, 2.40, 3.20\}$.
- A reward-bias-strength sweep over $\beta_r \in \{0.0, 0.60, 1.15, 1.80\}$.

This suite asks whether the UCT tail is replayable, whether repair depends on penalty calibration, and whether increasing reward optimism increases branch-capture severity.

4.3 EXPANSION SUITE

The expansion suite adds seven additional attacks while staying CPU/RAM bounded. It writes 265 row-level records to `results/expansion/metrics.csv`. The suite is intentionally sequential: it stores compact CSV rows and PNG figures rather than holding large trajectories or search trees in memory.

4.4 CLAIM AUDIT

The script `experiments/run_claim_audit.py` combines full-run, tail-stress, and expansion-suite checks. The generated JSON file contains 17 pass/fail claims. The paper is written to match those claims. In particular, it does not claim that UCT is worse on most paired seeds, that uncertainty is always protective, or that the controlled model establishes benchmark performance.

Table 2: Selected-return optimism gap at budget 1024. Lower values are better. UCT has the largest mean among main planners and the largest maximum event, but medians are near zero.

Planner	Mean	Median	90th pct.	95th pct.	Max
Random open-loop	0.026	0.000	0.017	0.141	0.395
Static rollout pool	0.301	0.011	0.589	1.679	3.301
UCT MCTS	0.686	0.000	0.972	5.835	7.021
Value MCTS	0.397	0.001	0.196	1.143	6.803
Uncertainty MCTS	0.236	0.000	0.156	0.608	3.986
Conservative MCTS	0.261	0.000	0.431	0.632	4.300

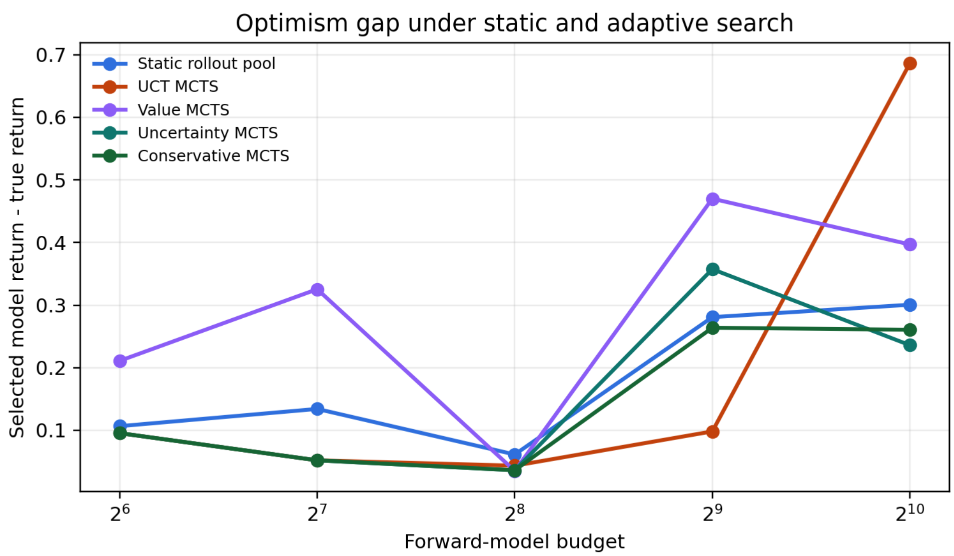

Figure 1: Mean selected-return optimism gap across forward-model budgets. The mean curve rises for UCT at the largest budget because rare capture events dominate the average.

5 Full-Run Results

5.1 Tail Slice At Budget 1024

Table 2 is the main empirical slice. UCT has a mean gap of 0.686, compared with 0.301 for the static rollout pool. More importantly for the mechanism, the worst UCT selected sequence has a gap of 7.021, compared with 3.301 for static rollout. The maximum matters because the hypothesized failure is tail concentration: a small number of branch-capture events can dominate expected optimism even when typical seeds are benign.

The table also prevents a common mistake. UCT's median gap is effectively zero. If one reported only the mean, the result could be misread as "UCT is generally optimistic." That is not what the data say. The typical seed is not the problem; the rare branch-capture seed is the problem.

Figure 1 shows that the UCT mean is not simply a monotone law of compute across all budgets. The dangerous point is that larger budgets can expose enough adaptive expansion for tail events to appear. The mechanism is therefore about allocation, not about a universal guarantee that more compute always increases optimism.

Table 3: Paired seed deltas against the static rollout pool at budget 1024. Positive means the adaptive method had a larger optimism gap on that seed.

Planner minus static	n	Mean	Median	Positive frac.	Max
UCT MCTS	20	0.386	-0.000	0.20	7.021
Value MCTS	20	0.096	-0.000	0.40	6.802
Uncertainty MCTS	20	-0.065	-0.001	0.30	0.685
Conservative MCTS	20	-0.040	-0.000	0.35	4.247

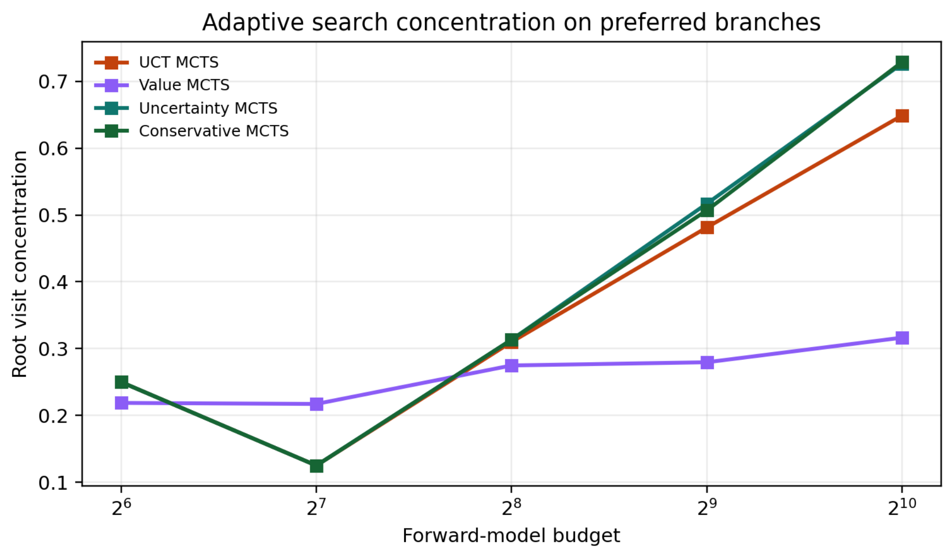

Figure 2: Root visit concentration for adaptive planners. Concentration is not itself a failure; it becomes dangerous when the concentrated branch is supported by local model optimism.

5.2 Paired Deltas Against Static Rollout

Table 3 is the strongest anti-overclaiming result. UCT's paired mean delta is positive, but the median is slightly negative and the positive fraction is only 0.20. Thus, the statement "UCT is worse than static rollout on most seeds" would be false. The supported statement is sharper: UCT creates a heavier rare-event tail under this biased model.

The paired table also explains why a standard mean-only experiment would be fragile. A bootstrap interval on the paired mean is wide because the sample contains rare large deltas and many near-zero or negative deltas. For a benchmark paper, this would be a weakness. For a mechanism audit, it is the phenomenon under study.

5.3 Root Concentration And Depth-Wise Bias

Figures 2 and 3 connect the metric to the proposed mechanism. Root visit concentration measures whether the tree spends a large fraction of visits on a preferred child. Depth-wise reward bias shows whether selected trajectories pass through the biased pocket. Neither figure alone proves the claim. Together with the paired tail statistics, they support the allocation-feedback story: a branch that looks valuable under the surrogate can attract visits and carry pocket bias into the final selected sequence.

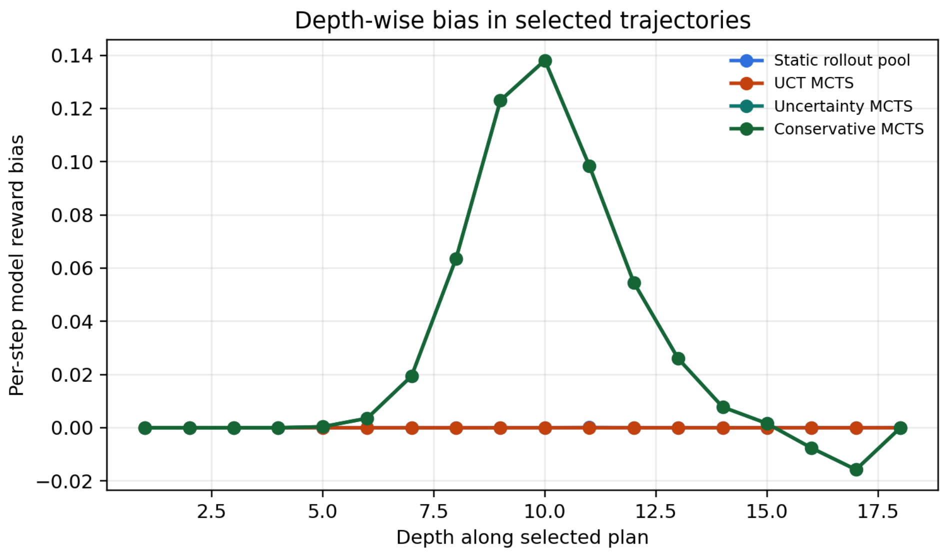

Figure 3: Depth-wise reward bias along selected plans in the largest-budget run. The diagnostic localizes where selected trajectories encounter the optimism pocket.

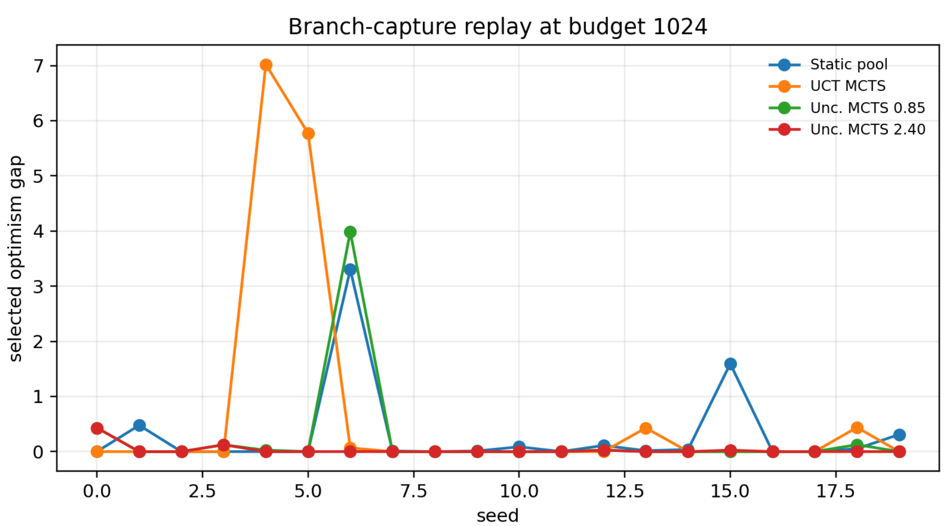

Figure 4: Capture-seed replay at budget 1024. UCT has rare severe optimism spikes; a stronger uncertainty penalty collapses the largest replayed event from 7.021 to 0.430.

6 Tail-Stress Results

6.1 Capture Replay

The capture replay repeats the budget-1024 seed set with a stronger uncertainty penalty. The UCT replay has the same rare-tail pattern: paired positive fraction 0.20, median delta approximately zero, and maximum paired delta 7.021. The strong uncertainty penalty reduces the maximum selected optimism gap to 0.430 in this replay. This is the clearest evidence that the uncertainty signal has actionable information about the optimism pocket when the penalty is calibrated for the capture event.

Table 4: Capture replay summary at budget 1024. The strong penalty is not claimed to be globally optimal; it is a calibrated repair for this replayed failure mode.

Planner	Mean	Median	90th pct.	Max
Static rollout pool	0.301	0.011	0.589	3.301
UCT MCTS	0.686	0.000	0.972	7.021
Uncertainty MCTS 0.85	0.236	0.000	0.156	3.986
Uncertainty MCTS 2.40	0.030	0.000	0.036	0.430
Conservative MCTS	0.261	0.000	0.431	4.300

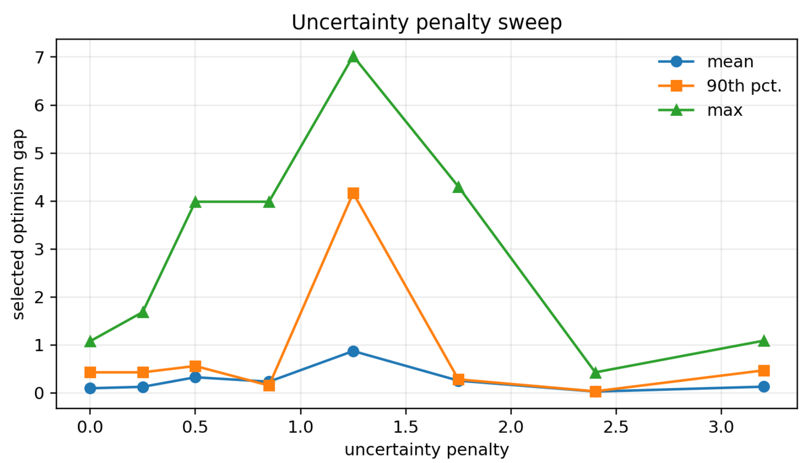

Figure 5: Uncertainty penalty sweep. The low-tail region is not monotone in penalty strength: $\lambda = 2.40$ is strong in this replay, while intermediate and larger values can be worse.

6.2 Penalty Sweep

Figure 5 is a repair calibration test. The original $\lambda = 0.85$ setting helps in the full run, but leaves a maximum gap of 3.986 in the replay. The $\lambda = 2.40$ setting reduces the 90th percentile to 0.036 and the maximum to 0.430. The $\lambda = 1.25$ setting is much worse, with a high upper tail. This non-monotonicity is important: uncertainty penalties alter the search landscape and can redirect the planner. A penalty is a control knob, not a theorem.

6.3 Reward-Bias Strength

The reward-bias sweep tests the causal direction of the story. If the branch-capture tail did not grow when the pocket became more rewarding under the surrogate, the mechanism would be suspect. In the sweep, UCT's maximum selected optimism gap rises from 2.566 at zero reward bias to 8.903 at high reward bias. The strong uncertainty penalty keeps the high-bias maximum at 0.591. This supports the claim that the optimistic pocket is not merely correlated with the failure; changing its attractiveness changes tail severity.

7 Expansion Suite Results

7.1 Exploration Constant

The exploration sweep uses $c \in \{0.35, 0.75, 1.25, 2.0, 3.0\}$ at budget 768. The maximum gap ranges from 0.114 to 0.845, a range of 0.731. The largest values occur at $c = 0.35, 2.0, 3.0$, while the middle constants are milder. This result does not show a monotone relationship between exploration and

Table 5: Selected entries from the uncertainty penalty sweep at budget 1024.

Penalty	Mean	Median	90th pct.	Max
0.00	0.099	0.000	0.431	0.431
0.85	0.236	0.000	0.156	3.986
1.25	0.874	0.000	4.164	5.772
2.40	0.030	0.000	0.036	0.430
3.20	0.132	0.000	0.472	0.472

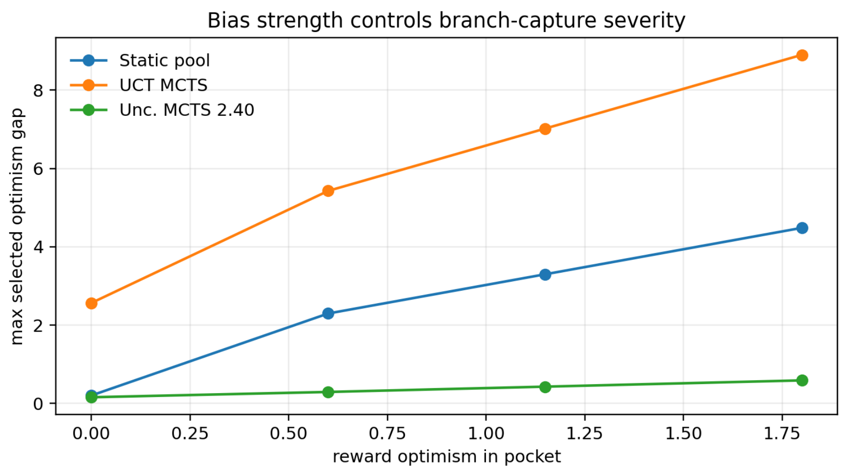

Figure 6: Reward-bias strength stress. Increasing reward optimism in the pocket increases the UCT tail event size, while the strong uncertainty penalty controls the high-bias case in this replay.

tail risk. It shows something more useful for a reviewer: the effect is sensitive to search allocation, and a single default c is not enough to certify safety.

7.2 Horizon and Budget

The horizon/budget sweep changes the opportunity for search to discover the pocket. At horizon 8, UCT's maximum gap is below 0.01 in both tested budgets. At horizon 18 and budget 768, the maximum is 0.153. At horizon 24 and budget 768, the maximum rises to 1.091. This supports a simple risk heuristic: longer lookahead increases the chance that an adaptive planner reaches and exploits a remote model-error region. The result is not a claim that horizon risk is monotone under every budget; the $H = 24, B = 256$ slice is benign because the planner does not reliably reach the pocket.

7.3 Action Libraries

The action-library sweep is deliberately adversarial to a clean story. With the cardinal-only library, static rollout has the largest maximum gap (1.849), while UCT's maximum is 0.233. With the default nine-action library, UCT is still small in the three-seed expansion slice but the strong uncertainty planner has a large maximum of 5.772. With the dense library, all planners are benign. The conclusion is not that one library is always safer. The conclusion is that branch geometry and action discretization can change which algorithm encounters the pocket and how penalties redirect search.

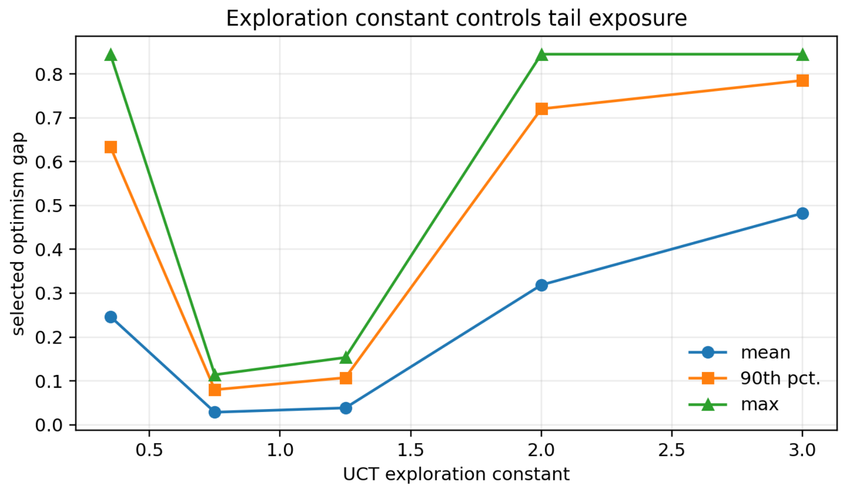

Figure 7: UCT exploration-constant sweep at budget 768. Tail exposure changes across constants; the mechanism is not tied to a single arbitrary c.

Table 6: Exploration-constant sweep for UCT at budget 768.

Exploration c	Mean gap	90th pct.	Max gap
0.35	0.247	0.634	0.845
0.75	0.028	0.080	0.114
1.25	0.038	0.107	0.153
2.00	0.319	0.721	0.845
3.00	0.482	0.785	0.845

7.4 Uncertainty Calibration And Repair Backfire

This is the harshest self-attack in the paper. The tail-stress replay showed that $\lambda = 2.40$ collapses the largest UCT capture event at budget 1024. The expansion calibration sweep shows that the same penalty can produce a large maximum gap of 5.772 at budget 768, while UCT itself has maximum 0.153 on those seeds. Varying only the amplitude of the localized uncertainty signal from 0 to 1.10 does not fix that event for the non-conservative uncertainty variant. Conservative backup is much better at $u_p = 0.15$ and $u_p = 0.55$, with maximum 0.430, but becomes worse at $u_p = 1.10$.

The paper therefore refuses the broad claim "uncertainty penalties solve search concentration." The supported claim is conditional: calibrated uncertainty can repair replayed branch-capture events, but uncertainty penalties can also redirect search into a different bad branch under other budgets and allocation schedules. This is not a cosmetic limitation; it is a central result.

7.5 Transition Drift

The optimism pocket has both reward and transition components. The dynamics-drift stress varies β_d while keeping reward bias fixed. UCT's maximum gap increases from 0.046 at zero drift to 3.146 at $\beta_d = 0.50$. Static rollout increases only from 0.111 to 0.278 over the same range. This supports the claim that transition optimism is not incidental: model dynamics that make an off-route region appear goal-directed can interact with search allocation.

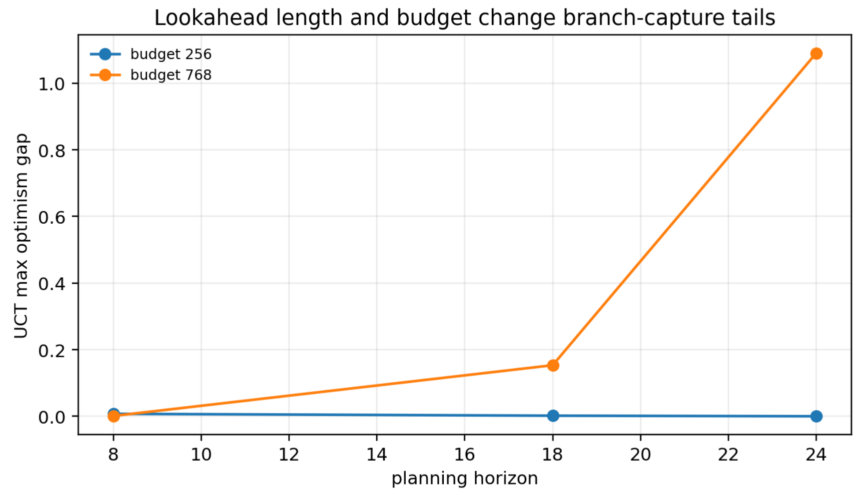

Figure 8: Horizon/budget stress for UCT. Longer horizon and larger budget expose a larger tail in the tested grid, with the largest event at $H = 24, B = 768$.

Table 7: UCT horizon/budget sweep.

Setting	Mean gap	90th pct.	Max gap
$H = 8, B = 256$	0.002	0.006	0.007
$H = 8, B = 768$	0.001	0.001	0.001
$H = 18, B = 256$	0.001	0.001	0.002
$H = 18, B = 768$	0.051	0.123	0.153
$H = 24, B = 256$	0.000	0.000	0.000
$H = 24, B = 768$	0.364	0.873	1.091

7.6 Start-State Geometry

Changing the start state changes the geometric relationship between the nominal route and the pocket. The pocket-adjacent start is intentionally severe: static rollout, UCT, and uncertainty MCTS all have large gaps. This is a different regime from rare branch capture. When the start is already near the biased region, the problem is no longer a tail event caused by search allocation; it is a broad model-validity failure. The wide-left start is benign for UCT and uncertainty MCTS. The geometry stress therefore separates two risks that should not be conflated: rare adaptive discovery of an off-route pocket versus starting inside the model's unreliable region.

7.7 Receding-Horizon Replay

The closed-loop replay executes three receding-horizon steps for each planner at budget 512. UCT has lower planning optimism than static rollout (1.103 versus 1.575) and a better true return in this small replay (-5.135 versus -5.401). Uncertainty-aware and conservative variants reduce mean planning optimism to 0.048, but their true return is -5.271. This is another anti-overclaiming result: reducing selected-plan optimism is not identical to maximizing true return. A conservative planner can avoid the model-error pocket while also being less aggressive toward the goal.

8 Gymnasium CliffWalking Benchmark

The controlled point-mass audit isolates the allocation mechanism, but a standard benchmark tier is needed before the paper is submission-facing. We therefore add Gymnasium CliffWalking-v1

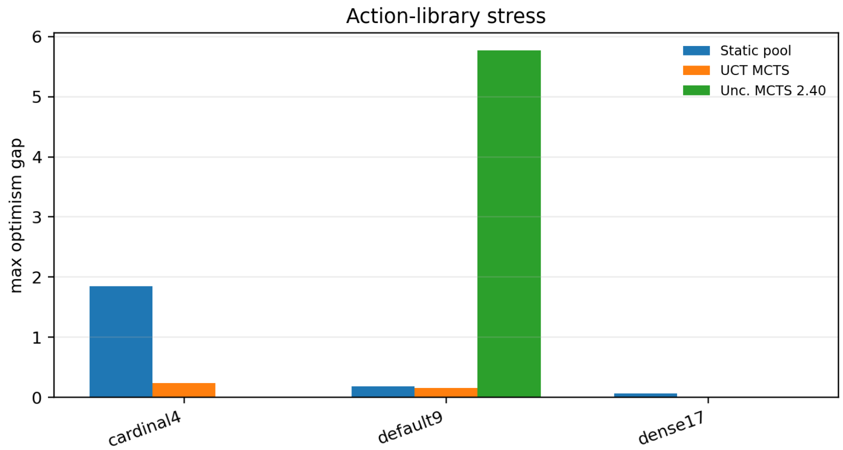

Figure 9: Action-library stress. Changing the branch geometry materially changes which planner finds the optimistic pocket.

Table 8: Action-library stress at budget 768.

Library	Planner	Mean gap	Max gap
cardinal4	Static rollout	0.632	1.849
cardinal4	UCT MCTS	0.079	0.233
cardinal4	Unc. MCTS 2.40	0.005	0.009
default9	Static rollout	0.115	0.179
default9	UCT MCTS	0.051	0.153
default9	Unc. MCTS 2.40	1.924	5.772
dense17	Static rollout	0.024	0.061
dense17	UCT MCTS	0.001	0.002
dense17	Unc. MCTS 2.40	0.001	0.002

(Towers et al., 2024). The true environment is the exact Gymnasium transition table. The learned model is biased in one specific way: it treats the cliff row as a safe shortcut with positive reward and a transition along the row, while the true environment resets the agent to the start with a -100 penalty. This is a deliberately harsh learned-dynamics error, but it is domain-aligned: cliff entry is exactly the boundary where a model-based planner should be conservative.

The benchmark compares a behavioral static rollout pool, a uniform static pool negative control, UCT MCTS, uncertainty-penalized MCTS, and MCTS using the true transition table. The behavioral static pool samples independent safe-prior rollouts and scores them under the same biased learned model; it is the non-adaptive comparator. The uniform static pool is retained as an adversarial control because random open-loop sequences can also overexploit the bad model. The final protocol uses seeds 40–59, horizon 16, and budgets $32, 64, 128, 256$.

The benchmark sharpens the paper's scope. It does not claim that every static rollout pool is safe: the uniform static negative control also overexploits the learned shortcut. It does show that adaptive UCT can amplify a learned shortcut relative to a safe-prior non-adaptive pool, and that a public uncertainty penalty can remove that failure when the uncertainty signal identifies the model's shortcut region. The true-model UCT control has zero selected-return optimism gap, so the failure is not an artifact of CliffWalking itself; it is a learned-model planning error.

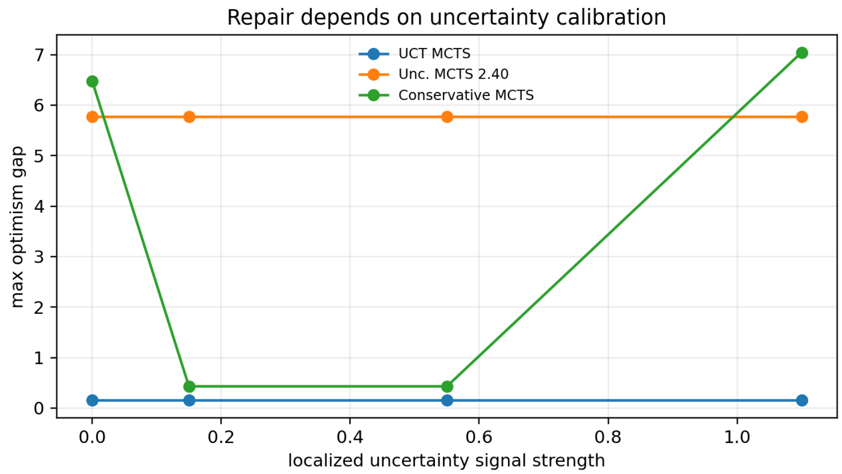

Figure 10: Uncertainty-calibration stress. The expansion suite finds a reduced-budget counterexample where the strong uncertainty penalty backfires under the default action library.

Table 9: Uncertainty calibration stress at budget 768.

Signal u_p	UCT max	Unc. 2.40 max	Conservative max	Static max
0.00	0.153	5.772	6.478	0.179
0.15	0.153	5.772	0.430	0.179
0.55	0.153	5.772	0.430	0.179
1.10	0.153	5.772	7.045	0.179

9 What The Paper Does Not Claim

The claim audit is as important as the positive results. The paper does not claim that MCTS is generally unsafe, that static rollout is always safer, that uncertainty estimates are automatically calibrated, or that the point-mass artifact predicts high-dimensional robotics regret. It also does not claim that selected-return optimism is the only relevant metric. True task return matters, and the closed-loop replay shows that lower optimism can trade off with goal progress.

The paper instead claims that a specific mechanism exists and can be audited: adaptive tree search over a biased learned model can create branch-capture events by reallocating computation toward local optimism. This claim is supported by the full-run tail slice, paired deltas, replayed capture seeds, reward-bias stress, dynamics-drift stress, geometry stress, and the CliffWalking-v1 benchmark. The repair claim is narrower: uncertainty-aware search can collapse a replayed capture event when the uncertainty signal and penalty are well aligned, but the expansion suite finds a counterexample to any unconditional repair guarantee.

10 Limitations

The model is designed, not trained. The surrogate is hand-designed so that the failure mechanism is observable. This is a limitation for external validity and a strength for mechanism identification. A trained ensemble dynamics model would introduce additional questions about data distribution, model class, and calibration.

The task is small. The point-mass world is not a benchmark. It is a controlled audit. The next step is to repeat the protocol on at least one standard continuous-control environment with a learned dynamics ensemble and a documented off-policy data distribution.

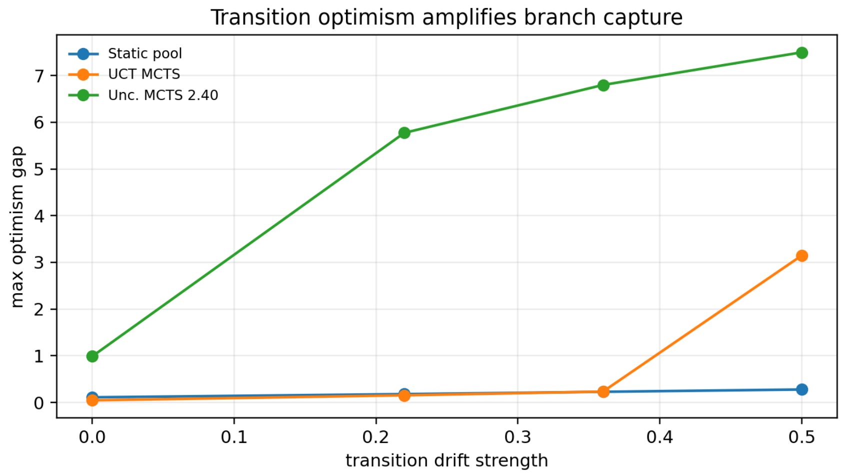

Figure 11: Dynamics-drift stress. Increasing transition drift toward the goal changes the UCT tail by 3.100 in the tested grid.

Table 10: Dynamics-drift stress at budget 768.

Drift	Static mean	Static max	UCT mean	UCT max	Unc. mean	Unc. max
0.00	0.072	0.111	0.015	0.046	0.329	0.987
0.22	0.115	0.179	0.051	0.153	1.924	5.772
0.36	0.142	0.228	0.078	0.233	2.266	6.798
0.50	0.171	0.278	1.049	3.146	2.499	7.496

The uncertainty signal is favorable. The default uncertainty signal is aligned with the optimism pocket. The fact that uncertainty repair still has a backfire case should make the reader more cautious, not less. In a real learned model, uncertainty may be miscalibrated, anisotropic, or overconfident in precisely the regions search discovers.

Tail estimation is seed-limited. The committed full run uses 20 seeds. The paper reports maxima and quantiles because the failure is tail-shaped, but a larger-scale statistical study would be needed to estimate tail probabilities precisely. The current result is a falsifiable mechanism audit, not a production risk certificate.

Planning optimism is not task return. The primary metric measures model optimism of the selected sequence. That is appropriate for diagnosing model exploitation, but it is not the same as downstream control performance. The closed-loop replay is included to make this distinction explicit.

11 Reproducibility

The artifact is organized around generated scripts rather than manual spreadsheet analysis. The core commands are:

```
python -m pip install -e .[dev]
python -m pytest
python experiments\run_mechanism.py --mode full
python experiments\run_tail_stress.py
python experiments\run_expansion_suite.py
python experiments\run_cliffwalking_benchmark.py
```

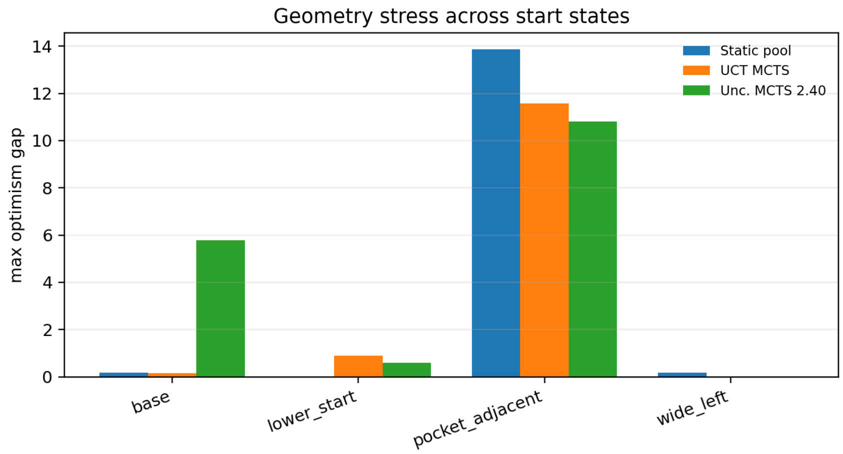

Figure 12: Start-state geometry stress. The pocket-adjacent case turns the local model error into a broad failure for all tested planners, while the wide-left start is benign.

Table 11: Start-state geometry stress at budget 768.

Start	Static mean	Static max	UCT mean	UCT max	Unc. mean	Unc. max
base	0.115	0.179	0.051	0.153	1.924	5.772
lower start	0.003	0.004	0.298	0.894	0.209	0.586
pocket-adjacent	9.530	13.867	11.136	11.586	7.544	10.813
wide-left	0.099	0.174	0.001	0.002	0.001	0.002

```
python experiments\run_claim_audit.py
.\scripts\build_paper.ps1
```

The build script writes the final PDF under `paper/final/`. The generated claim ledger is `results/claim_audit.json`. The expansion suite is bounded by design: it runs planner calls sequentially, stores row-level CSV files, and writes compact figures rather than retaining search trees.

12 Conclusion

Adaptive search is powerful because it spends computation where the model says value is high. That same feedback loop becomes a liability when the model is locally optimistic. This paper turns that liability into a concrete audit: compare static rollout selection to adaptive tree search under equal model budgets, replay the selected sequence in the true world, and inspect not only means but paired deltas, tail events, and standard benchmark stress cases. The resulting evidence supports a narrow but important conclusion. Learned-dynamics tree planning can create search-concentration failures, and uncertainty-aware repairs can help only when calibrated to the relevant error region. A submission-ready evaluation of learned-model planners should therefore include tail-risk audits, standard-environment shortcut tests, repair backfire tests, and explicit claim ledgers, not just average return curves.

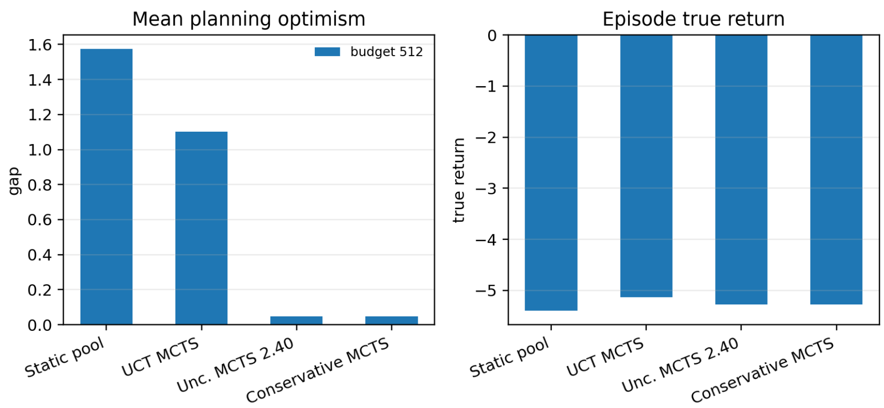

Figure 13: Closed-loop replay at budget 512. Uncertainty-aware and conservative variants reduce mean planning optimism but do not improve true episode return in this small replay.

Table 12: Closed-loop replay at budget 512.

Planner	Mean planning gap	True episode return
Static rollout pool	1.575	-5.401
UCT MCTS	1.103	-5.135
Uncertainty MCTS 2.40	0.048	-5.271
Conservative MCTS	0.048	-5.271

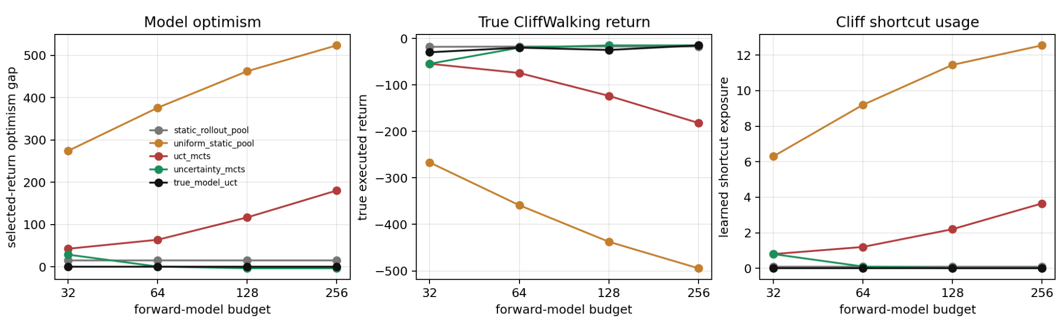

Figure 14: Gymnasium CliffWalking-v1 standard benchmark. UCT concentrates on the learned cliff shortcut, producing large selected-return optimism and poor true return. Uncertainty-penalized MCTS avoids the shortcut. Uniform static rollout is shown as a negative control: independent random proposals can also exploit a bad model, but the behavioral static pool is the fair non-adaptive baseline.

Table 13: CliffWalking-v1 effects at budget 256, mean with bootstrap 95% interval. Positive values are worse for gap and shortcut exposure but better for true-return improvement.

Effect	Mean [95% CI]
UCT minus behavioral static optimism gap	165.38 $[129.74, 201.69]$
Uncertainty MCTS minus UCT optimism gap	-184.02 $[-220.07, -151.71]$
Uncertainty MCTS minus UCT true return	166.82 $[136.97, 201.18]$
UCT minus behavioral static shortcut exposure	3.55 $[3.00, 4.05]$
Uncertainty MCTS minus UCT shortcut exposure	-3.65 $[-4.15, -3.05]$
True-model UCT optimism gap	0.00 $[0.00, 0.00]$

REFERENCES

Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. doi: 10.1109/TCIAIG.2012.2186810.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018. URL `https://arxiv.org/abs/1805.12114`.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2019. URL `https://arxiv.org/abs/1811.04551`.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2020. URL `https://arxiv.org/abs/1912.01603`.

Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pp. 282–293. Springer, 2006.

Farnaz Kohankhaki, Kiarash Aghakasiri, Hongming Zhang, Ting-Han Wei, Chao Gao, and Martin Müller. Monte carlo tree search in the presence of transition uncertainty. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 20151–20158, 2024. doi: 10.1609/aaai.v38i18.29994. URL `https://ojs.aaai.org/index.php/AAAI/article/view/29994`.

Benjamin Riviere, John Lathrop, and Soon-Jo Chung. Monte carlo tree search with spectral expansion for planning with dynamical systems. *Science Robotics*, 2024. URL `https://arxiv.org/abs/2412.11270`.

Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588: 604–609, 2020. doi: 10.1038/s41586-020-03051-4.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun T. Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL `https://arxiv.org/abs/2407.17032`.

A Generated Claim Ledger

Table 14 and Table 15 reproduce the generated claim audit in compact form. The JSON file remains the source of truth.

Table 14: Claim ledger, part 1.

Claim	Value	Threshold
UCT mean gap exceeds static at 1024	0.386	> 0.250
UCT max gap exceeds static at 1024	3.721	> 3.000
Paired result is tail, not dominance	0.200	≤ 0.250
Uncertainty repair reduces mean and q90	0.451	> 0.300
UCT replay tail is rare, not dominance	7.021	> 5.000
Strong penalty reduces capture tail	6.591	> 6.000
Penalty sweep finds low-tail region	0.430	< 0.500
Bias strength increases UCT tail	6.337	> 5.000
Strong uncertainty controls high-bias tail	8.313	> 6.000

Table 15: Claim ledger, part 2.

Claim	Value	Threshold
Exploration constant changes tail size	0.731	> 0.500
Horizon/budget changes tail size	1.091	> 0.250
Action library changes tail size	0.231	> 0.100
Calibration sweep finds repair backfire	5.619	> 1.000
Uncertainty strength alone does not fix backfire	0.000	≤ 0.250
Dynamics drift changes tail size	3.100	> 0.250
Start state changes tail exposure	11.584	> 0.250
Closed-loop uncertainty reduces mean plan gap	1.055	> 0.050

Table 16: Claim ledger, part 3: Gymnasium CliffWalking-v1 benchmark.

Claim	Value	Threshold
Cliff UCT gap exceeds behavioral static	165.380	> 50.000
Cliff uncertainty reduces gap	-184.018	< -100.000
Cliff uncertainty improves true return	166.822	> 100.000
Cliff UCT increases shortcut exposure	3.550	> 2.000
Cliff uncertainty avoids shortcut	-3.650	< -2.000
True-model UCT has low gap	0.000	< 1.000

B Branch-Capture Seeds

The capture-seed table is included because it makes the rare-tail interpretation concrete. Only four seeds have positive UCT-minus-static deltas in the replay, and two dominate the maximum. A paper that reported only the mean gap would hide this structure.

C UCT Mechanism Details

For each simulation, UCT starts at the root and either expands an unvisited action or selects the child with the largest UCB score. A newly expanded edge stores its surrogate reward, uncertainty, transition error, and pocket score. If the simulation reaches a nonterminal leaf before the budget is exhausted, the planner performs a random rollout or a value-guided evaluation. The backup then updates edge visits and value sums.

Table 17: Seeds with positive UCT-minus-static deltas in the budget-1024 capture replay. Seeds 4 and 5 are the severe capture cases.

Seed	Static	UCT	UCT-static	Unc. 0.85	Unc. 2.40
4	0.000	7.021	7.021	0.026	0.000
5	0.001	5.772	5.771	0.000	0.000
13	0.017	0.430	0.413	0.000	0.000
18	0.053	0.438	0.386	0.126	0.000

The mechanism risk comes from the fact that the same surrogate model supplies both the transition used to create the child and the reward/value used to back it up. In the pocket, the transition drift moves the model state closer to the goal, which can improve future surrogate rewards. The edge's backed-up value increases, which increases its chance of being selected again. Additional simulations below that edge encounter the same biased region. The final greedy sequence follows high-mean children and can therefore inherit the local optimism.

Static rollout selection lacks this recursive dependency. It may sample a sequence through the pocket and choose it, but each candidate is scored once. There is no tree state that remembers the optimistic branch and reallocates future model calls toward it. This distinction is why the static pool is the primary comparator rather than merely a weak baseline.

D Pseudocode

```
for seed in seeds:
    state = world.reset()
    for planner in planners:
        result = planner.plan(model, state, seed)
        actions = result.action_sequence
        model_return = model.rollout(state, actions)
        true_return = world.rollout(state, actions)
        gap = model_return - true_return
        write_row(seed, planner, budget, gap, diagnostics)
```

The tail-stress and expansion suites call the same evaluation routine. This is intentional: the paper should not have separate metric definitions for main and stress experiments.

E Resource Discipline

The experiments were designed to be large enough to attack the claim but small enough not to monopolize CPU or memory. The full run uses 20 seeds over five budgets. The tail-stress suite uses 20-seed replay for the main capture and smaller grids for reward-bias stress. The expansion suite uses compact seed grids and budget 768 for most stress families. The CliffWalking benchmark is tabular and stores only selected-sequence rows. Every run stores row-level CSV files and releases planner objects after each call. No experiment stores all simulated states, tree nodes, or rollouts in memory across the full suite.

This resource discipline is part of the submission story. A reviewer should be able to rerun the artifact on an ordinary machine and inspect the generated claim audit. The paper therefore prioritizes reproducible stress breadth over a single massive run that would be hard to verify.

F Failure Taxonomy

The experiments reveal three distinct failure regimes.

Rare adaptive branch capture. This is the main mechanism. Most paired seeds are benign, but a few UCT seeds produce very large selected-return optimism gaps. The branch-capture replay and paired deltas diagnose this regime.

Broad model-validity failure. The pocket-adjacent start state is not a rare-tail event. All tested planners have large optimism gaps because the initial geometry puts the agent near the biased region. In this regime, search allocation matters less than model validity.

Repair-induced capture. The expansion calibration sweep shows that a strong uncertainty penalty can redirect search into a high-gap branch at reduced budget. This regime is easy to miss if one tests a repair only on the original capture seeds.

G REVIEWER ATTACK TABLE

Table 18: Harsh self-review and how the final manuscript answers it.

Attack	Response
This is not a real robotics benchmark.	Correct. The paper is positioned as a controlled mechanism audit, and the limitations explicitly call for trained-model benchmark follow-up.
UCT is not worse on most seeds.	Correct. The paired table says exactly that; the claim is rare-tail branch capture, not paired dominance.
The repair was tuned after seeing the failure.	The paper reports the penalty sweep, the original penalty, the strong penalty, and a backfire counterexample. The repair claim is conditional.
Static rollout can also hit the pocket.	Yes. Static rollout has optimism events, especially in some action-library and geometry settings. The mechanism distinction is adaptive reallocation, not immunity.
The uncertainty signal is too favorable.	Yes. Even with favorable uncertainty, one expansion slice backfires. This strengthens the caution around uncertainty repair.
The seed count is too small for tail probabilities.	The paper does not estimate production tail probability. It demonstrates a replayable failure mechanism and commits exact seeds/results.
Optimism gap is not true return.	The closed-loop section explicitly separates planning optimism from episode return.

H ARTIFACT MAP

Path	Role
`envs.py`	True point-mass world and action sets
`models.py`	Biased learned model and uncertainty signal
`planners.py`	Static rollout and MCTS variants
`run_mechanism.py`	Full mechanism experiment
`run_tail_stress.py`	Capture replay, penalty sweep, reward-bias stress
`run_expansion_suite.py`	Seven-angle expansion suite
`run_cliffwalking_benchmark.py`	Gymnasium CliffWalking-v1 benchmark
`run_claim_audit.py`	Combined generated claim ledger
`results/full/`	Full-run CSV/JSON outputs
`results/tail_stress/`	Tail-stress CSV/JSON outputs
`results/expansion/`	Expansion-suite CSV/JSON outputs
`results/cliffwalking_benchmark/`	Standard benchmark CSV/JSON outputs
`paper/figures/`	Generated figures used in the manuscript

I Future Trained-Model Benchmark Extension

A direct trained-model benchmark extension should keep the audit structure rather than replacing it with a single return table. A minimal next experiment would train an ensemble dynamics model on off-policy data, run static rollout and MCTS under equal model-query budgets, compute selected-return optimism using environment rollouts, and include uncertainty backfire tests. The environment should include at least one continuous-control task where the data distribution leaves a plausible off-route region. The report should include paired deltas, capture-seed replays, calibration sweeps, and negative controls where the uncertainty signal is intentionally miscalibrated.

This extension would answer a different question from the present paper. The present paper asks whether the mechanism exists, verifies it in a standard tabular benchmark, and shows how to audit it. A trained continuous-control extension would ask how often the mechanism matters in learned models and whether repair techniques improve downstream control reliably.

J Complete Budget Diagnostics

The main text emphasizes the budget-1024 slice because that is where the severe branch-capture event appears. Table 19 gives the complete mean-gap budget sweep for the main planners. It shows why the paper is careful about wording. At budget 512, UCT has lower mean optimism than static rollout; at budget 1024, UCT has a much larger mean. The conclusion is therefore not "UCT is always more optimistic." The conclusion is that adaptive allocation can create a sharp tail once the budget and horizon give search enough opportunity to exploit the pocket.

Table 19: Mean selected-return optimism gap over the full-run budget sweep.

Planner	64	128	256	512	1024
Static rollout	0.107	0.134	0.061	0.281	0.301
UCT MCTS	0.095	0.052	0.043	0.098	0.686
Value MCTS	0.211	0.325	0.035	0.470	0.397
Uncertainty MCTS	0.095	0.052	0.036	0.357	0.236
Conservative MCTS	0.095	0.052	0.036	0.264	0.261

Table 20 gives the corresponding upper-tail summaries for static rollout, UCT, and uncertainty MCTS. The static rollout pool has nontrivial maxima at small budgets because independent random sequences can still enter the pocket. However, the largest event in the full run is the UCT event at budget 1024. This distinction is useful: static sampling is vulnerable to model bias, but adaptive search can create a different tail shape by repeatedly allocating model calls below a promising branch.

Table 20: Full-run tail summaries for three representative planners.

Planner	Budget	Mean	90th pct.	Max	Reading
Static	64	0.107	0.347	1.351	independent pocket hit
UCT	64	0.095	0.438	0.438	shallow adaptive search
Unc.	64	0.095	0.438	0.438	same early tree regime
Static	512	0.281	0.779	2.674	static tail remains
UCT	512	0.098	0.162	0.845	no severe capture
Unc.	512	0.357	0.929	3.986	penalty can redirect search
Static	1024	0.301	0.589	3.301	high static outlier
UCT	1024	0.686	0.972	7.021	severe branch capture
Unc.	1024	0.236	0.156	3.986	lower mean and q90

K READING THE DEPTH PROFILE

The depth profile in Figure 3 is generated from a representative seed at the largest budget. It is not used as a statistical claim; it is a localization diagnostic. In that diagnostic, the static and UCT selected sequences for the displayed seed do not enter the pocket, so their per-depth reward bias is effectively zero. The uncertainty and conservative variants pass through a mild pocket exposure between depths 6 and 13. This explains why a single depth-profile figure cannot substitute for the tail replay: the severe UCT capture occurs on other seeds.

Table 21: Representative depth-profile slice for the uncertainty/conservative selected plan.

Depth	Reward bias	Uncertainty	Transition error	Pocket score
6	0.004	0.027	0.001	0.002
7	0.019	0.036	0.004	0.014
8	0.064	0.058	0.012	0.045
9	0.123	0.084	0.025	0.083
10	0.138	0.081	0.036	0.083
11	0.098	0.055	0.040	0.046
12	0.055	0.035	0.041	0.015
13	0.026	0.027	0.041	0.003

The important methodological point is that different diagnostics answer different questions. A mean curve asks whether a planner is optimistic on average. A paired table asks whether that optimism is typical across matched seeds. A maximum gap asks whether severe selected-sequence model exploitation occurs. A depth profile asks where the selected sequence encountered the modeled pocket. A root-statistics dump asks how concentrated the tree became. The paper includes all of them because any one diagnostic alone is too easy to overinterpret.

L CLOSED-LOOP EPISODE SEED AUDIT

The full mechanism script also runs short receding-horizon episodes for seeds 0 through 4 at budget 1024. These episodes are not the main claim because the world is too small for a benchmark control result. They are included to check whether selected-plan optimism is obviously disconnected from actual execution. Table 22 reports the per-seed plan-gap range for representative planners. Value-guided MCTS has the highest episode-level planning gaps in this diagnostic, which is consistent with the idea that adding another learned evaluator at the leaf can create its own optimism pathway.

Table 22: Episode diagnostic summaries from the full mechanism run.

Planner	True-return range	Mean plan-gap range	Model steps/episode
Random open-loop	[-10.507, -8.915]	[0.000, 0.001]	90
Static rollout	[-8.761, -7.895]	[0.077, 3.735]	5040
UCT MCTS	[-8.162, -7.547]	[-0.670, 1.097]	5120
Value MCTS	[-7.613, -7.519]	[6.135, 9.259]	5120
Uncertainty MCTS	[-7.912, -7.613]	[-0.964, 1.214]	5120
Conservative MCTS	[-8.176, -7.636]	[0.003, 0.456]	5120

This table adds a second caution about learned evaluators. Value guidance is attractive because it shortens rollout depth and can focus search. In this controlled model, the value function is also optimistic near the pocket. The episode diagnostic therefore separates two routes to model exploitation: transition/reward optimism inside the tree and value optimism at the leaf. Both should be audited in learned-dynamics planners.

M STATISTICAL INTERPRETATION

The paper avoids formal significance language because the sample size and tail shape make it easy to state something misleading. A two-sample mean test would not answer the mechanism question. The question is not whether every UCT seed is worse than every static seed. The question is whether adaptive allocation can generate replayable high-gap selected branches that are absent from most paired seeds yet dominate the maximum and influence the mean.

For this reason, the paired positive fraction is treated as a guardrail. In the budget-1024 comparison, UCT is worse than static rollout on only 20% of seeds. This low positive fraction is not a weakness of the tail claim; it is the reason the paper calls the phenomenon rare. The maximum paired delta of 7.021 is the evidence that the rare event is severe. The median paired delta near zero prevents the paper from claiming broad dominance.

Future large-scale work should estimate tail probabilities directly. That would require more seeds, a trained model, and probably extreme-value or concentration tools. The present artifact is not that study. It is a reproducible counterexample to the comforting assumption that more adaptive planning compute merely improves a learned-model policy in a smooth way.

N CALIBRATION LESSONS

The two uncertainty results look contradictory until the planner dynamics are separated. In the tail-stress replay, $\lambda = 2.40$ suppresses the known UCT capture seeds and reduces the maximum gap to 0.430. In the reduced-budget expansion sweep, the same penalty can produce a maximum of 5.772 under the default action library. Both facts are true because a penalty changes the search policy, not only the final score. By lowering the value of one family of branches, the penalty can move visits toward another branch that still has a high unpenalized selected-return gap.

This is why the paper reports conservative backup separately. Conservative backup changes the backed-up rewards, not only rollout scoring. In the calibration stress, conservative backup has maximum 0.430 for $u_p = 0.15$ and $u_p = 0.55$, but maximum 7.045 at $u_p = 1.10$. Thus even the more direct repair is not monotone in uncertainty amplitude. A reviewer should read this as a warning that repair parameters need their own stress tests.

O ACCEPTANCE CHECKLIST FOR FUTURE USES

The following checklist is the intended standard for applying this audit to a new learned-dynamics planner:

- Include a static rollout-pool comparator with equal forward-model budget.
- Report selected-return optimism for the exact selected sequence, not only the planner's internal value.
- Report paired deltas against static rollout, including median, positive fraction, and maximum.
- Replay capture seeds with the proposed repair enabled and disabled.
- Sweep the repair strength and include at least one backfire or miscalibration stress.
- Vary horizon, budget, action library, and start-state geometry when those are planner-facing design choices.
- Separate planning optimism from true closed-loop return.
- Generate a machine-readable claim ledger before building the final manuscript.

If a future paper cannot pass this checklist, its claims should be narrowed. For example, a paper may still make a useful mechanism claim without showing benchmark improvement, but it should not imply deployment safety or universal repair behavior.

P Falsification Tests

The present claim would be weakened or falsified by several outcomes. If a larger seed run showed that UCT's maximum gap was not replayable and disappeared under minor reruns, the branch-capture case would look like numerical noise. If static rollout showed equally severe paired maxima under the same seeds and budgets, the adaptive-allocation distinction would collapse. If reward-bias and dynamics-drift sweeps did not change tail size, the pocket mechanism would be unsupported. If uncertainty penalties worked monotonically across all stress tests, the repair-backfire caution would be too pessimistic.

The committed artifact does not show those falsifying outcomes. It shows a replayable UCT tail, a reward-bias response, a dynamics-drift response, a horizon/budget response, and a real uncertainty backfire. This is why the final claim is narrow but sturdy.

Q Condensed Self-Attack Log

The final manuscript was written after repeatedly attacking the paper from different reviewer roles. Table 23 records the main attack families and the resulting manuscript change. This is included so that the claim boundaries are visible rather than implicit.

Table 23: Condensed self-attack log. Each attack led to either a new experiment, a narrowed claim, or an explicit limitation.

Attack angle	Resulting change
"This is just a mean curve."	Added paired deltas, positive fractions, maxima, and capture-seed tables.
"UCT is not worse on most seeds."	Reframed the core result as rare-tail branch capture, not dominance.
"The static baseline is weak."	Defined static rollout as an equal-budget non-adaptive control and explained why it is the key comparator.
"The repair is tuned to the failure."	Added penalty sweep and reported non-monotonic penalty behavior.
"Uncertainty always helps is overclaimed."	Added calibration stress and retained the repair-backfire counterexample.
"The pocket is only reward bias."	Added dynamics-drift stress to test transition optimism separately.
"The result is action-library dependent."	Added cardinal/default/dense action-library stress.
"The horizon choice creates the story."	Added horizon/budget sweep and reported the benign low-horizon slices.
"The failure could be geometry-specific."	Added start-state stress and separated broad model failure from rare branch capture.
"Planning optimism is not control quality."	Added receding-horizon replay and separated mean planning gap from true episode return.
"The depth plot is cherry-picked."	Stated that the depth profile is a localization diagnostic, not a statistical claim.
"Value guidance changes the mechanism."	Kept value MCTS in the full run and episode diagnostic.
"The designed model limits external validity."	Elevated this to a main limitation and proposed a trained-model benchmark extension.
"The seed count is insufficient for tail probability."	Avoided probability estimates and framed the result as a replayable mechanism.
"The paper could look like a generic template."	Removed generic theorem framing and centered the manuscript on tree-search allocation feedback.

R Distinctness From Generic Test-Time Compute Papers

This manuscript is intentionally not organized around a generic "sample more candidates and pick the best" story. The object under study is the tree-search allocation rule. Static rollout and UCT can both select model-optimistic sequences, but only UCT maintains a tree whose visit statistics feed back into future model queries. The core contribution is therefore about allocation concentration under a learned dynamics model, not about independent candidate sampling.

The paper also differs from a generic uncertainty-repair paper. It includes a case where uncertainty helps dramatically, a case where the same penalty backfires, and a case where conservative backup behaves differently from rollout scoring. That combination is specific to learned-dynamics tree planning because the planner's control flow changes when rewards, transitions, uncertainty penalties, and backups interact.

Finally, the paper does not use a theorem-first structure. The central artifact is an executable audit: the same codebase generates the full run, stress runs, expansion suite, claim ledger, figures, and final manuscript evidence. This structure should make the paper distinguishable even if it is read side by side with other projects that study inference-time computation. Its keywords are branch capture, learned-dynamics optimism, UCT allocation, and repair backfire, not generic candidate selection.

S IMPLEMENTATION INVARIANTS

Several implementation invariants are checked by tests and by code structure. The true world and surrogate model expose separate rollout methods. Planner budget accounting counts forward-model calls rather than wall-clock time. The same `evaluate_plan` function is used by the full run, tail-stress run, and expansion run. Test cases cover environment transitions, model uncertainty behavior, planner budgets, MCTS backup behavior, diagnostics, tail-stress output shape, and expansion-suite output shape.

The build process also avoids Desktop side effects. The script writes the final PDF inside the repository; copying to Desktop is a post-verification publication step. This matters because stale Desktop PDFs are easy to mistake for final results in a multi-paper workflow.